ChatGPT

Mapbox Token Failing to Initialize in Production – Root Causes & Solutions

Root Cause Analysis

1. Build-Time Env Variable Not Embedded: In production, the Mapbox access token wasn't being injected into the app bundle due to how Vite handles environment variables. Vite **replaces** `import.meta.env` **at build time**, not at runtime [1]. In the code, the token is obtained via a dynamic environment lookup (`getEnv('MAPBOX_TOKEN')`), which checks `import.meta.env` keys at runtime [2]. This dynamic access means the token was **not baked into the production build**. If the Cloudflare Pages environment variable wasn't correctly provided (e.g. wrong name or missing `VITE_` prefix), `getMapboxToken()` would return an empty string [3], leaving `mapboxgl.accessToken` unset. The team attempted to fix this by using Vite's `define` to hard-code the token at build time [4]. However, if the Cloudflare env var was misconfigured (e.g. set as `MAPBOX_TOKEN` instead of `VITE_MAPBOX_TOKEN`) or not present, the define fallback used a default token string [5]. In production, that default token may have been invalid (expired or not authorized for the domain), still causing the "invalid Mapbox access token" error.

2. Multiple Token Injection Paths (Timing Conflict): The implementation used *several redundant methods* to inject the token, which in production might have interfered with each other:

- **Meta Tag & Global Window:** The HTML includes a meta tag with the token and a script setting `window.__MAPBOX_TOKEN__` **before** loading Mapbox GL JS [6]. After loading the Mapbox script, another snippet assigns `mapboxgl.accessToken` from `window.__MAPBOX_TOKEN__` [7]. This ensures the **global** Mapbox script gets the token early.

- **React Component Initialization:** Separately, the React code imports Mapbox GL and runs an IIFE to set the token on the *imported* `mapboxgl` instance at module load [8], also copying it to `window.__MAPBOX_TOKEN__` for good measure. It logs a preview and warns if the token format is wrong [9].

These parallel approaches indicate the token is set on both the global `window.mapboxgl` (from the CDN script) and the bundler's `mapboxgl` module. In development this may go unnoticed, but in **production** the timing and duplication can cause issues. For example, bundling Mapbox GL separately could lead to two instances of the library. If the React app uses the bundled instance, it might not see the token set by the global script. Conversely, if the CDN script runs later, it might overwrite or use a different token. In short, the token initialization was not unified. The code even includes a last-resort check right before creating the map: if no valid token is present, it assigns a hardcoded token one more time [10]. All of this complexity suggests a race or configuration problem where **in production the token remained undefined or incorrect at the moment of map initialization**, despite these safeguards.

3. React Production Build vs Dev Timing: In development, the app likely loads all modules immediately with Vite's dev server (and perhaps the token from a local `.env` was inlined). In the production build (React 18 + Vite), code-splitting and lazy loading may delay when the `Map.jsx` module (and its token-setting IIFE) executes. For example, if the map is on a route that isn't the initial page, the Map component's code might load on demand. It's possible the external Mapbox script was loaded and the map started rendering (or threw an error) **before** the React code set the token. The multiple chunk strategy in `vite.config.js` explicitly separates Mapbox (`manualChunks` for "mapbox-core") [11] [12], which could affect when the code runs. Any mismatch in timing could mean `mapboxgl.accessToken` wasn't set on the correct instance in time. (Notably, the production build also strips out `console.log` by default [13], so any warning like "[MapboxGL] Token missing" might not appear, making it harder to catch in prod.)

4. Cloudflare Pages Environment Delivery: Cloudflare Pages only provides environment variables at **build time** (for front-end use) – there is no runtime injection into static front-end apps. This means the Mapbox token must be present during `npm run build`. The project's use of `loadEnv` and Vite `define` shows awareness of this [4], but if the deployment process didn't correctly supply the env vars, the build fell back to a placeholder. Indeed, an internal script suggests they had to manually set Pages env vars via Wrangler CLI [14] [15]. Any misstep there (like not running the script, or an env file not being loaded) would result in a build with an empty or wrong token. In short, the **production build likely lacked the real token string in the code**. As a consequence, `mapboxgl.accessToken` remained blank, and Mapbox's API requests returned 401 Unauthorized. (A Reddit discussion on Vite envs emphasizes that *"Vite doesn't dynamically read environment variables at runtime…everything is baked into the build."* [1] If the token wasn't baked in, the client had nothing valid to use.)

5. Mapbox Token Scope/Domain Restrictions: Another environment-level factor is Mapbox token restrictions. A token can be domain-restricted. If the token used in development was unrestricted or limited to `localhost`, but the production site's domain wasn't added, Mapbox will reject requests with a 403 Forbidden despite the token string being correct. The error message in the app ("invalid access token") is generic and would appear in this case too. According to Mapbox's docs, a 403 error can occur if *"a request is called from an unauthorized URL"* when the token has URL restrictions [16]. Similarly, if the token was inadvertently using the wrong scopes (not all read scopes), it could fail to load maps [17]. Given the troubleshooting steps taken, it's likely the token was meant to be public and valid, but **it's worth confirming** that the deployed domain is added to the token's allowed URLs (or that no URL restrictions are set). A stale or revoked token (if the default token was an old one) would also behave this way [18].

In summary, the Mapbox map failed in production because the app wasn't actually providing a valid token to `mapboxgl` when running on Cloudflare Pages. The likely causes were a combination of **environment variable misconfiguration (token not injected at build)**, **duplicative token-setting logic that was brittle in the production build**, and possibly **Mapbox token restrictions** that only manifested on the live site. All these led Mapbox to treat the token as invalid, resulting in no map tiles loading.

Recommendations and Robust Fixes

To resolve these issues, a more streamlined and reliable token injection approach is needed. Below are recommended patterns and fixes, with code examples:

1. Inject the Token at Build Time (Single Source of Truth): Ensure the Mapbox token enters the client app via a build-time substitution, using Vite's environment handling. The simplest method is to **use Vite's** `import.meta.env` **directly** in your code, with the token configured as a `VITE_` prefixed variable in Cloudflare Pages.

- **Set the env var in Cloudflare Pages:** In your Pages project settings (or via Wrangler), add `VITE_MAPBOX_TOKEN` with your token. This makes it available during the build.

- **Use it in code:** For example, in the map initialization module (or a config file), do:

```
mapboxgl.accessToken = import.meta.env.VITE_MAPBOX_TOKEN;
```

This static reference lets Vite replace it with the actual token string at build time (or `""` if none was provided) [1]. It avoids any runtime guessing. If you have a config utility, update it to reference `import.meta.env.VITE_MAPBOX_TOKEN` explicitly. In fact, the project's new `environmentConfig.js` maps `MAPBOX_TOKEN` to exactly that key [19] and checks it in order [20]. Using `environmentConfig.getMapboxToken()` would leverage that static approach and also fall back to the meta tag if present [21] [22].

- **Remove dynamic token logic for client:** You can simplify `getMapboxToken()` to trust the build-time env. For example:

```
export const getMapboxToken = () => import.meta.env.VITE_MAPBOX_TOKEN ||
"";
```

Then call this once at startup. The numerous fallbacks (window vars, meta tag, etc.) can be reduced once the build is injecting the right value.

2. Consolidate Token Setting (avoid race conditions): Pick **one clear place** to set `mapboxgl.accessToken` and do it early. The current code sets it in HTML *and* in React module IIFE *and* right before `new mapboxgl.Map()`. This can be simplified:

- In a React app, a good pattern is to set the token **before creating the Map instance**, but after your app has the value. For example, in your top-level `<App>` or wherever the map component mounts, ensure you call something like:

```
import mapboxgl from 'mapbox-gl';
import { getMapboxToken } from './utils/environmentConfig'; // uses
import.meta.env under the hood
mapboxgl.accessToken = getMapboxToken();
```

Do this once (the IIFE in `Map.jsx` was attempting this). If your map is in a lazily loaded route, you might instead set a global token in a small `<script>` in `index.html` using the injected env (see next point).

- **Use a meta tag OR global, not both:** The meta tag approach is actually fine for security (it keeps the token out of visible JS code). If you prefer it, continue using `<meta name="mapbox-token" content="...">` in `index.html`, but populate the content via an environment variable. For instance, you could put a placeholder in `index.html` and replace it during the build. One approach is to leverage a templating plugin or simply a post-build script. Example:

```
<!-- index.html -->
<meta name="mapbox-token" content="%MAPBOX_TOKEN%">
```

and then have a build step replace `%MAPBOX_TOKEN%` with the token. The project's **MCP server/agent** and the script `fix-mapbox-token.js` [23] [24] suggest they intended to automate this. Using such a script (or even a simple sed command in CI) can inject the real token into the meta tag before deployment. Then `getMapboxToken()` should first check the meta (as it already does) [25]. This guarantees that by the time any JS runs, the token is in the DOM for retrieval. *Important:* ensure the token is HTML-escaped properly if doing string replacement.

- **Alternatively, use a global JS variable:** If not using meta, setting a global like `window.__MAPBOX_TOKEN__` early in `index.html` works too [26]. But make sure to remove duplicate assignments. In production, you only need to set it once. For example:

```
<script>
  window.__MAPBOX_TOKEN__ = "<%= VITE_MAPBOX_TOKEN %>";
  // Immediately set global mapboxgl token if library is already loaded:
  if (window.mapboxgl) {
    window.mapboxgl.accessToken = window.__MAPBOX_TOKEN__;
  }
</script>
```

(Here `<%= VITE_MAPBOX_TOKEN %>` implies using a templating mechanism to inject the env value). Then your React code can simply do `mapboxgl.accessToken = window.__MAPBOX_TOKEN__` if needed, or even rely on the global having been set. The key is to not have conflicting values. In the current setup, the `window.__MAPBOX_TOKEN__` in the HTML was actually correct, but the React bundle may have overridden `mapboxgl.accessToken` with an empty string if its env was empty. Unifying these ensures the token is consistent.

3. Avoid Double-Loading Mapbox GL JS: It's risky and unnecessary to include Mapbox GL via both the CDN script and the bundled module. **Choose one approach**:

- **Using Bundled Mapbox GL:** Remove the `<script src="https://api.mapbox.com/mapbox-gl-js/v3.4.0/mapbox-gl.js"></`

`script>` from `index.html` to avoid loading a second copy. Let Vite handle it via the import (`import mapboxgl from 'mapbox-gl'`). Ensure the CSS is still included (you can keep the Mapbox CSS link or import the CSS in JS as done in `Map.jsx` [27]). This way, there is a single `mapboxgl` instance. Set its accessToken as described above. The manual chunk config in Vite will output a separate file for Mapbox, but that's fine – it will load when needed. Make sure the token is set before any `new mapboxgl.Map()` call.

- **Using CDN (External) Mapbox GL:** If you prefer not to bundle the library (to leverage CDN caching or avoid bundle size), mark `mapbox-gl` as an external dependency in Vite. You can do this by adjusting the Rollup config, e.g.:

```
// vite.config.js
export default defineConfig({
  // ...,
  build: {
    rollupOptions: {
      external: ['mapbox-gl']
    }
  }
});
```

This will prevent Vite from including Mapbox GL in the bundle. Instead, you *must* have the CDN script in the HTML. In that case, modify your React code to use the global `window.mapboxgl` (or ensure that the import uses the global – some libraries check for a global). You might do:

```
const mapboxgl = window.mapboxgl;
mapboxgl.accessToken = window.__MAPBOX_TOKEN__;
```

after the script is loaded. One way is to move the map initialization into a function that runs after the script load (the HTML already sets the token right after loading the script [7]). The current setup was *close* to this, but the presence of the import meant a second instance. Choosing one eliminates the mismatch.

4. Verify Cloudflare Pages Env Configuration: On Cloudflare Pages, environment variables for front-end must be set for the build. Double-check that in your Pages dashboard, `VITE_MAPBOX_TOKEN` **is defined for the production environment** (and any preview environments). The fact that a custom deployment script was used [15] implies the normal route might have been bypassed. If using Git-based deploys, add the variable in the UI. If using Wrangler CLI, pass the `--var VITE_MAPBOX_TOKEN="xyz"` during build or use a `.env` file that the build reads. You had `.env.production` in the repo – ensure it contains `VITE_MAPBOX_TOKEN` (prefix required) and that Vite actually loads it. Calling `loadEnv(mode, process.cwd(), "")` loads all env vars including system ones [28], so it should capture it if present. After a successful build, **inspect the output**: the token should appear (e.g., grepping the built JS for `pk.ey` should find your key). If it doesn't, the build is still missing it.

5. Address Mapbox Token Restrictions: Now that the token will be correctly provided, make sure it's authorized for the production usage:

- **Scopes:** Use the Mapbox account dashboard to ensure the token has all required scopes for reading maps (usually all public scopes for styles, tiles, etc.) [17] . An incomplete scope can cause 403 errors.

- **URL (Domain) Restrictions:** If you have added URL restrictions to the token, include your production site's hostname. On Cloudflare Pages, that might be a `<your-project>.pages.dev` domain or a custom domain. If you're unsure, you can remove URL restrictions on the token for testing. According to Mapbox docs, a token with unauthorized URL will yield a 403 error [16] ("Forbidden"), which would manifest as an "invalid token" message in the map. Aligning the token's allowed URLs with your deployment will fix this.

- If the previous default token was from a different Mapbox account or an old project ("hardworkco" is in the token string seen in code [6]), consider generating a fresh token in your Mapbox account for this project to avoid confusion. Update the Cloudflare env to use the new token. This also ensures you know exactly what restrictions/scopes it has.

6. Improve Diagnostics in Production: To avoid blind spots, implement some runtime checks specifically for the production build:

- **Console Confirmation:** It's okay to log a short message on startup confirming the token was set (e.g., `console.log("Mapbox token prefix:", mapboxgl.accessToken?.slice(0,6));`). During debugging, you might disable the `drop_console` in terser for one build to see these logs. You already log a token preview in dev [29] ; ensure it isn't stripped out or wrap it in a `if (import.meta.env.DEV)` condition to keep it in non-prod only.

- **Map Error Handling:** The code uses `map.on('error', ...)` to catch token errors and display a user-friendly message [30] . This is good. In addition, you could programmatically detect a missing token at startup and `console.error` a clear message. For example, right after setting `mapboxgl.accessToken`, do:

  ```
  if (!mapboxgl.accessToken) {
    console.error("No Mapbox token found – check configuration!");
  }
  ```

  This will show up if the token wasn't set at all.

- **Network Monitoring:** In the browser dev tools, inspect the network requests to Mapbox. If you see requests returning 401/403, check the request URL – it often includes `access_token=<token>` as a query param. This can reveal if the token is blank or incorrect in production. With the fixes above, you should see your actual token in those URLs. (You may want to rotate the token if it was exposed in failed attempts, unless it's meant to be public.)

- **Consistent Testing:** Use the provided test pages (like `/map-error-test` or the minimal map test) after deploying fixes. These pages were created to isolate map issues. For instance, *MinimalMapTest.jsx* directly used `import.meta.env.VITE_MAPBOX_TOKEN` in one version [31]. Ensuring that works in production is a good litmus test. If that page reports success (or logs the token properly), your env injection is working.

By implementing the above changes, the Mapbox token should reliably initialize in production. In practice, many of these steps boil down to **simplifying the solution**: let build-time environment provide the token once, use that source consistently, and remove extraneous fallbacks. This reduces the surface for bugs.

Further Insights and Patterns

The challenges here reflect common pitfalls in modern front-end deployments:

- **Environment Variables in SPAs:** As noted, frameworks like Vite only include variables prefixed with `VITE_` by default, and they must be present during build [1]. One cannot rely on `process.env` at runtime in a static site – there is no Node process in the browser. Always use build-time injection or explicitly embed needed config into the app (or fetch it from an API).

- **Dynamic vs Static Env Access:** The initial `environmentMapper.js` tried to dynamically check many names (VITE_MAPBOX_TOKEN, MAPBOX_TOKEN, MAP_API_TOKEN, etc.) [32]. While thorough, this approach can fail for front-end code because those variations won't exist unless statically replaced. The refactored `environmentConfig.js` addresses this by explicitly referencing `import.meta.env.VITE_MAPBOX_TOKEN` [33], which allows Vite to substitute the value. The lesson is to use **explicit environment keys** in front-end code so that your bundler can do its job.

- **Multiple Mapbox GL Instances:** Including the library twice can lead to subtle issues (e.g., one instance not having the token or event handlers registering on one instance and not the other). It's generally advisable to have a single source. If you ever suspect this in other contexts, check `mapboxgl.version` in console from both the imported and global object – if you get two different values or objects, you know there's duplication.

- **Cloudflare Pages vs Workers:** Cloudflare Pages is great for static sites but doesn't support secure runtime secrets for front-end (since everything is public after build). If you ever needed to hide the token (e.g., if it were a secret key), you'd have to move logic to a Cloudflare Worker or use Pages Functions to inject it server-side. In this case, Mapbox tokens are public keys, so baking it in is fine. The provided documentation indicates a plan to migrate to Workers for more control [34] [35]. In a Workers setup, you could store the token as a secret and inject it via a middleware that sets `window.ENV` or modifies the response HTML. But again, for a public API token, that complexity isn't needed – simpler is better.

Implementing these fixes will ensure the Mapbox map loads correctly in production. The token will be available at the right time and place, and the Cloudflare environment will deliver it consistently. With a valid token initialized, the map should render without the dreaded "invalid access token" error, and your fallback error boundary (which shows a static map image or message) won't be triggered except in genuine error cases.

Lastly, remember to remove any temporary debugging artifacts (like extra logs or hardcoded tokens) once the issue is resolved. Keeping the token management clean and centralized will make future maintenance much easier [36] [37]. Good luck, and happy mapping!

Sources:

- Project Code – Mapbox token initialization and config logic [38] [4] [2]
- Project Code – HTML template with token injection points [39] [7]
- Project Code – Fallback and error handling for missing token [10] [30]
- Mapbox Documentation – Token error causes (unauthorized domain, scopes) [17] [16]
- Vite Environment Handling – Env vars are inlined at build (discussion) [1]

---

[1] Environment variable not working during production deployment : r/reactjs
https://www.reddit.com/r/reactjs/comments/1jhs439/environment_variable_not_working_during/

[2] [32] environmentMapper.js
https://github.com/JW-Flo/AWhittleWandering/blob/40ee49b5baa64c7f378d9ca144b1449e2e5c06b5/shared/config/environmentMapper.js

[3] [25] mapboxConfig.ts
https://github.com/JW-Flo/AWhittleWandering/blob/40ee49b5baa64c7f378d9ca144b1449e2e5c06b5/shared/mapbox/mapboxConfig.ts

[4] [5] [11] [12] [13] [28] vite.config.js
https://github.com/JW-Flo/AWhittleWandering/blob/40ee49b5baa64c7f378d9ca144b1449e2e5c06b5/48Continental_Starter/public-site/vite.config.js

[6] [7] [26] [39] index.html
https://github.com/JW-Flo/AWhittleWandering/blob/40ee49b5baa64c7f378d9ca144b1449e2e5c06b5/48Continental_Starter/public-site/index.html

[8] [9] [10] [27] [29] [30] [38] Map.jsx
https://github.com/JW-Flo/AWhittleWandering/blob/40ee49b5baa64c7f378d9ca144b1449e2e5c06b5/48Continental_Starter/public-site/src/components/Map.jsx

[14] [15] set-cloudflare-env.cjs
https://github.com/JW-Flo/AWhittleWandering/blob/40ee49b5baa64c7f378d9ca144b1449e2e5c06b5/48Continental_Starter/public-site/set-cloudflare-env.cjs

[16] [17] [18] Troubleshooting Access Tokens | Help | Mapbox
https://docs.mapbox.com/help/troubleshooting/token-errors/

[19] [20] [21] [22] [33] environmentConfig.js
https://github.com/JW-Flo/AWhittleWandering/blob/40ee49b5baa64c7f378d9ca144b1449e2e5c06b5/48Continental_Starter/public-site/src/utils/environmentConfig.js

[23] [24] fix-mapbox-token.js
https://github.com/JW-Flo/AWhittleWandering/blob/40ee49b5baa64c7f378d9ca144b1449e2e5c06b5/scripts/fix-mapbox-token.js

[31] MinimalMapTest.jsx
https://github.com/JW-Flo/AWhittleWandering/blob/40ee49b5baa64c7f378d9ca144b1449e2e5c06b5/48Continental_Starter/public-site/src/components/MinimalMapTest.jsx

34 35 PAGES_TO_WORKERS_MIGRATION.md

https://github.com/JW-Flo/AWhittleWandering/blob/40ee49b5baa64c7f378d9ca144b1449e2e5c06b5/docs/PAGES_TO_WORKERS_MIGRATION.md

36 37 CODE_AUDIT_FIXES.md

https://github.com/JW-Flo/AWhittleWandering/blob/40ee49b5baa64c7f378d9ca144b1449e2e5c06b5/48Continental_Starter/public-site/docs/CODE_AUDIT_FIXES.md